PicoRX Firmware

Code Guide for Modification | Raspberry Pi Pico / Pico 2 SDR Receiver

Source analysed: PicoRX-master (dawsonjon/PicoRX) - September 2026

1. What this code is

PicoRX is the firmware for a direct-conversion software-defined radio (SDR) receiver covering 0-30 MHz (LW, MW, SW) built around a Raspberry Pi Pico (RP2040) or Pico 2 (RP2350, ARM or RISC-V). The analog front end is a Tayloe-style quadrature sampling detector (QSD): an analog switch driven by a quadrature local oscillator produced by the Pico's PIO, followed by an op-amp. The Pico digitises the resulting I/Q baseband with its internal ADC, does all filtering and demodulation in fixed-point C++, and outputs audio as PWM on a GPIO (plus USB audio). It supports AM, synchronous AM, LSB, USB, narrow-band FM and CW, with a 128x64 OLED user interface, an optional 320x240 ILI934x TFT for a waterfall / SSTV image decoder, 512 memory channels, and Kenwood-style CAT control over USB serial.

Key design facts to keep in mind when modifying:

- **Dual core.** Core 0 runs the user interface, tuning, CAT and TFT; core 1 runs the real-time receive DSP loop. They communicate only through two structs guarded by a semaphore.
- **Tuning is done partly by changing the system clock.** The LO frequency comes from a PIO state machine whose clock divider is fractional; to hit a frequency closely, the code also picks one of ~110 PLL settings between 150 and 200 MHz. The residual error is removed digitally.
- **Fixed-point, time-critical DSP.** Hot functions are tagged `__not_in_flash_func` so they execute from RAM. All audio processing happens in blocks of 64 output samples.
- **Settings live in flash.** Both the auto-saved state and the memory channels are C arrays placed in flash and rewritten at runtime with a specific core-lockout procedure.

2. Repository layout

Path / file(s)	Purpose
`picorx.cpp`	Entry point: creates global objects, launches core 1, runs the core-0 cooperative scheduler.
`rx.h / rx.cpp`	Receiver 'engine': hardware setup (ADC, DMA, PIO NCO, band pins), settings hand-off, core-1 streaming loop, USB audio feed.
`rx_dsp.h / rx_dsp.cpp`	All per-sample DSP: CIC decimation, DC/IQ correction, frequency shift, demodulators, AGC, squelch, tone controls, spectrum & scope capture.
`rx_definitions.h`	Global constants: sample rates, block sizes, FFT sizes, mode IDs, S-meter reference levels.
`fft.*, fft_filter.*, noise_reduction.*, cic_corrections.*`	Fixed-point FFT, overlap FFT channel filter (bandwidth, sideband selection, noise reduction, auto-notch), CIC droop correction.

Path / file(s)	Purpose
`nco.pio, nco.cpp/.h, clocks.h, clocks_pico.cpp, clocks_pico2.cpp`	PIO quadrature LO and the system-clock/PLL search table used to set the LO frequency.
`quadrature_si5351.*`	Optional external Si5351 quadrature LO over I2C (enabled from HW Config).
`pwm_audio_sink.*`	PWM audio output on GPIO16 with DMA ping/pong and 16x interpolation.
`usb_audio_device.c, usb_descriptors.*, tusb_config.h, ring_buffer_lib.*`	TinyUSB composite device: USB audio (demodulated audio or raw IQ) + CDC serial.
`ui.h / ui.cpp`	OLED UI (largest file, ~2750 lines): display views, menus, scanners, memory store/recall, splash.
`settings.h / settings.cpp`	User-facing settings structs, defaults, translation into `rx_settings`, flash save/restore.
`autosave_memory.cpp, memory.cpp (+ .h)`	Flash-resident arrays: 512 auto-save slots and 512 memory channels (factory contents).
`cat.h / cat.cpp`	CAT (Kenwood TS-480-like) command parser over USB serial.
`waterfall.*, ili934x.*`	Auxiliary TFT display driver, waterfall renderer, SSTV image display.
`codecs/decode_sstv.*, codecs/cordic.*`	SSTV decoder (Martin, Scottie, PD, SC2 modes).
`button.*, encoder.h, rotary_encoder.*, button_encoder.*, quadrature_encoder.pio`	Input devices: debounced buttons, PIO quadrature rotary encoder or button-based 'encoder'.
`ssd1306.*, font_*.h, fonts.h, gfxfont.h, logo.h`	Pixel drawing into the u8g2 frame buffer, fonts, logo bitmap.
`pins.h`	Single source of GPIO assignments.
`battery_check.cpp`	Separate small utility firmware that reports battery voltage.
`memory_loader/`	Python tools to upload/download memory channels over USB serial from a CSV file.
`simulations/, grc/`	Python/C++ design scripts and GNU Radio flowgraphs used to derive DSP constants.
`utils/`	Font generation, screen-dump decoder, autosave helper scripts.
`PCB/, enclosure/, user_manual/`	KiCad hardware, 3D-printable enclosure, user manual (RST + PDF).
`external/u8g2`	Git submodule (u8g2 graphics library) - must be fetched with `git submodule update`.

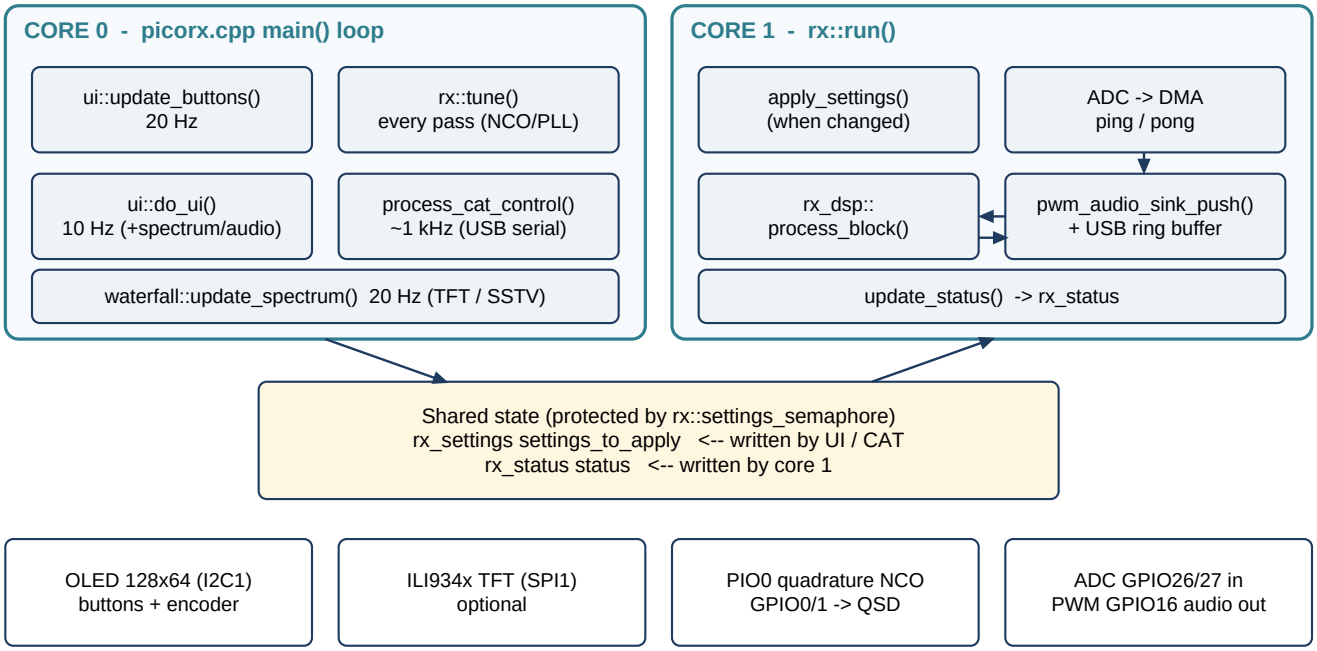
3. Build system

The project uses the Pico SDK with CMake (`CMakeLists.txt`). If `PICO_SDK_PATH` is not set in the environment, CMake fetches SDK 2.1.0 from GitHub automatically. The u8g2 library is compiled as a static library from the submodule. Target names depend on the board:

CMake arguments	Executable produced	Clock table
`-DPICO_BOARD=pico`	`picorx.elf/.uf2` (+ `battery_check`)	`clocks_pico.cpp`
`-DPICO_BOARD=pico2 -DPICO_PLATFORM=rp2350-arm-s`	`pico2rx` (+ `battery_check_pico2`)	`clocks_pico2.cpp`
`-DPICO_BOARD=pico2 -DPICO_PLATFORM=rp2350-riscv`	`pico2rx-riscv`	`clocks_pico2.cpp`
`-DBUTTON_ENCODER=1` (optional, any board)	Adds `BUTTON_ENCODER` define; uses `button_encoder.cpp` instead of `rotary_encoder.cpp`	-

Other build details: C++ is compiled with `-fno-rtti -fno-exceptions`; `-Wall -Werror` is enabled, so new warnings stop the build. Stdio goes to USB (UART disabled). PIO headers are generated from `nco.pio` and `quadrature_encoder.pio`. The `build_all` script builds all three variants and zips the UF2 files. A new source file must be added to the `PICORX_SRCS` list. For Visual Studio / VS Code, `.vscode/settings.json` configures the CMake Tools extension (build dir `build`, Release, `PICO_BOARD=pico`).

4. Runtime architecture



4.1 Start-up (picorx.cpp)

Global objects are constructed before `main()` in this order: `rx_settings settings_to_apply`, `rx_status status`, `rx receiver` (configures ADC, DMA, PIO NCO, band pins, PWM audio), `waterfall waterfall_inst` (SPI TFT) and `ui user_interface` (OLED, buttons, encoder). `main()` then turns on the LED, enables a 2-second watchdog, launches `core1_main()` (which calls `receiver.run()`), creates a high-priority alarm pool for USB audio, and calls `user_interface.autorestore()` to load the last saved settings.

4.2 Core 0 scheduler

The core-0 loop is a simple time-sliced scheduler based on `time_us_32()`. Change the rates with the `#defines` at the top of `picorx.cpp`.

Task	Rate	What it does
`watchdog_update()`	every pass	Keeps the 2 s watchdog alive; anything that blocks core 0 for longer resets the Pico.
`ui.update_buttons()`	20 Hz (BUTTONS_REFRESH_US)	Samples and debounces buttons (and the button-encoder).
`receiver.tune()`	every pass	Applies frequency / ppm / IF changes to the NCO (and Si5351). Runs on core 0 so I2C stays on one core.
`ui.do_ui()` + `get_spectrum/get_audio`	10 Hz (UI_REFRESH_HZ)	UI state machine, draws the OLED, pulls spectrum and scope data from DSP.
`process_cat_control()`	~1 kHz (CAT_REFRESH_US)	Reads USB serial, parses CAT commands.
`waterfall.update_spectrum()`	20 Hz	Draws the TFT waterfall or runs the SSTV decoder display.

4.3 Core 1 receive loop (rx::run)

`rx::run()` initialises USB audio, starts a repeating timer (~0.53 ms) that services TinyUSB, and then loops forever:

- If `settings_changed` is set, call `apply_settings()` (pushes all DSP settings into `rx_dsp`).
- Read battery and temperature ADC channels (only possible while streaming is stopped).
- Start the ADC in round-robin mode over inputs 0 and 1 (I and Q) at 480 kS/s total, with two chained DMA channels filling `ping_samples` / `pong_samples` (2048 samples each, ~4.3 ms).
- Inner loop: wait for ping, `process_block()`, push 64 audio samples to PWM; wait for pong, repeat. `update_status()` copies signal strength, filter config, USB buffer level etc. into `rx_status`.
- Break out and restart when settings change, when the receiver is suspended (e.g. for flash writes), or periodically to refresh battery/temperature.

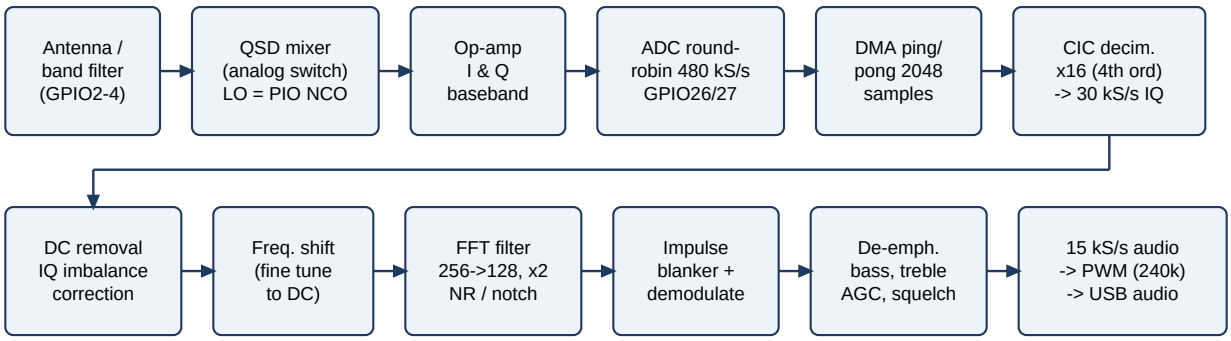
4.4 How settings reach the receiver

There are three layers of settings. Understanding this chain is the single most useful thing when adding features:

Layer	Type / where	Owner
User settings (persisted)	`s_settings` = `s_channel_settings` + `s_global_settings` (settings.h). Held as `ui::settings`.	UI (core 0); saved to flash
Receiver request	`rx_settings settings_to_apply` (rx.h) - runtime values in engineering units.	Written by `apply_settings_to_rx()` and CAT
Receiver status	`rx_status status` (rx.h) - signal dBm, busy time, battery, filter config, USB level.	Written by core 1 in `update_status()`

```
UI changes ui::settings  -->  ui::apply_settings()  -->  apply_settings_to_rx()   (settings.cpp)
    receiver.access(changed)          // sem_acquire_blocking + set settings_changed
    rx_settings.x = f(settings.y) ... // translate every field
    receiver.release()                // sem_release
core 1:  if(settings_changed) rx::apply_settings()  -->  rx_dsp_inst.set_xxx(...)
core 0:  rx::tune()  (every loop) -> nco_set_frequency() / external_nco.set_frequency_hz()
```

5. Signal chain and DSP



Everything from 'CIC decim.' onward runs in rx_dsp::process_block() on core 1.

5.1 Key constants (rx_definitions.h)

Constant	Value	Meaning
`adc_sample_rate`	480 000	Total ADC rate, alternating I and Q (240 kS/s each).
`decimation_rate`	32	Overall ADC-to-audio decimation (CIC x16 then FFT filter x2).
`audio_sample_rate`	15 000	Demodulated audio rate.
`pwm_audio_sample_rate`	240 000	PWM update rate (audio interpolated x16).
`fft_size / new_fft_size`	256 / 128	Forward / inverse FFT sizes in the channel filter (bin width ~117 Hz).
`adc_block_size`	2048	ADC samples per DMA block -> 64 audio samples per block.
`cic_order`	4	CIC filter order; `extra_bits` = 1 kept after bit-growth removal.
`AM, AMSYNC, LSB, USB, FM, CW`	0-5	Mode IDs (mirrored by `e_mode` in settings.h and the UI strings).
`S0, S1, S9, S9_10`	-127, -121, -73, -63 dBm	S-meter reference levels.

5.2 Local oscillator and tuning

The PIO program in `nco.pio` is a 4-instruction loop that outputs the 2-bit Gray sequence 0,1,3,2 on GPIO0/GPIO1, giving two square waves 90 degrees apart at one quarter of the state-machine clock. `nco_set_frequency()` (nco.cpp) searches every entry in `possible_frequencies[]` (clocks_pico*.cpp, each a PLL setting between 150 and 200 MHz) for the combination of system clock and 8-bit-fraction PIO divider that lands closest to 4 x (tuned frequency +/- IF offset). It then reprograms the PLL and the PIO divider and returns the frequency actually achieved.

- The difference between wanted and achieved frequency, `offset_frequency_Hz`, is passed to `rx_dsp::set_frequency_offset_Hz()` and removed by a digital mixer (`frequency_shift()`, 2048-entry sine table with a phase accumulator).
- The IF offset (`if_frequency_hz_over_100`, default 4.5 kHz) keeps the wanted signal away from DC; `if_mode` chooses upper (1), lower (0) or nearest (2).
- Frequency calibration: `ppm` scales the tuned frequency before the search.
- Because the system clock changes, the PWM audio wrap value is recalculated on every retune via `pwm_audio_sink_update_pwm_max()`. `disable_pwm()/enable_pwm()` (tuning_option 0/1/2) control how the audio output behaves during the clock change to limit clicks.
- External NCO: when `enable_external_nco` is set, the PIO pins are released, the system clock is fixed at the first table entry, and a Si5351 at I2C address 0x60 on the OLED bus (I2C1) generates the LO.

5.3 Band-select outputs

`rx::apply_settings()` drives GPIO2/3/4 (`PIN_BAND_0..2`) as a 3-bit code chosen by comparing the tuned frequency with seven thresholds `band_N_limit x 125 kHz`. Defaults (settings.h) are 0x02...0x80, i.e. 250 kHz, 500 kHz, 1, 2, 4, 8 and 16 MHz. Above band 7 the code is 000; below band 1 it is 111. Thresholds are editable in HW Config > Bands; the code mapping itself is the if/else ladder at the top of `rx::apply_settings()`.

5.4 rx_dsp::process_block() step by step

Stage	Function / location	Notes for modifying
De-interleave I/Q	top of `process_block`	Even samples are I, odd are Q; `swap_iq` flips them.
CIC decimate x16	`decimate()`	4 integrators + 4 combs on I and Q; output ~30 kS/s complex.
DC removal	static accumulators in `process_block`	Mean over 2048 samples subtracted from I and Q.
IQ imbalance correction	`iq_imbalance_correction()`	Adaptive gain/phase correction, enabled by `iq_correction`.
Fine frequency shift	`frequency_shift()`	Removes NCO error + IF offset so the wanted signal sits at DC.
Channel filter x2	`fft_filter::process_sample()`	Hann-windowed 256-pt overlap FFT, zero bins outside `start_bin..stop_bin`, optional noise reduction and auto-notch, 128-pt inverse FFT. Also fills the spectrum capture buffer.
Magnitude / phase	`rectangular_2_polar()` (utils.cpp)	Fast approximations used by AM, FM, S-meter.
Impulse blanker	`apply_impulse_blanker()`	Threshold ratio 3.0-2.0 relative to average magnitude.
Demodulate	`demodulate()`	AM: magnitude minus DC. AMSYNC: 3rd-order PLL (30 Hz). FM: phase difference. LSB/USB: I channel (sideband chosen in filter). CW: mix with sidetone oscillator.
Tone shaping	`apply_deemphasis/bass/treble()`	Simple IIR filters; settings are step indices.
AGC	`automatic_gain_control()`	Leaky max-hold with attack/hang/decay; manual gain when AGC = Manual.
Squelch	`squelch()`	Signal amplitude vs threshold table (S0..S9+30 dB) with timeout.
Outputs	end of `process_block`	64 audio samples; scope capture; optional raw IQ to USB ring buffer; average signal amplitude.

Bandwidth table

`rx_dsp::set_mode(mode, bw)` looks up the filter's stop bin (bin width ~117 Hz) from this table; edit it to change filter widths. For SSB the start bin and sideband flags are set in the same function.

Bandwidth	AM	AMS	LSB	USB	NFM	CW
0 very narrow	19	19	16	16	31	0
1 narrow	22	22	19	19	34	1
2 normal	25	25	22	22	37	2
3 wide	31	31	25	25	40	3
4 very wide	63	63	28	28	43	4

AGC table

Set in `rx_dsp::set_agc_control()` (factors are right-shift amounts; timer counts are at 15 kS/s). AGC Gain (0-60 dB in 6 dB steps) sets `manual_gain = 1 << agc_gain`, which is the fixed gain in Manual mode and the maximum

gain the AGC may apply in the other modes:

Setting	Decay (s) / factor	Attack (s) / factor	Hang
0 fast	0.151 / 10	0.001 / 2	0.1 s (1500)
1 medium	0.302 / 11	0.001 / 2	0.25 s (3750)
2 slow	0.604 / 12	0.001 / 2	1 s (15000)
3 long ('very slow')	2.414 / 14	0.001 / 2	2 s (30000)
4 manual	-	-	fixed gain

5.5 Audio output paths

- `pwm_audio_sink.cpp`: each audio sample is scaled by the volume gain (table in `rx::apply_settings()`, 0-9 in 3 dB steps), interpolated x16 and sent by DMA to the PWM compare register of GPIO16. Its return value is used to measure DSP busy time (shown on the status page).
- USB: `rx::process_block()` duplicates mono audio to stereo and pushes it to a ring buffer that the TinyUSB audio callback drains. In 'Raw IQ' mode (menu USB Stream) the decimated I/Q pairs are streamed instead. Host volume/mute arrive through `on_usb_set_mutevol()`.

6. User interface (ui.cpp)

The UI is a non-blocking state machine: `ui::do_ui()` is called at 10 Hz and every menu function returns `true` when it is finished, `false` while still active, keeping its own state in `static` variables. Nothing in the UI may block for long because core 0 also services the watchdog.

6.1 Top-level states

State	Entered by	Behaviour
`splash`	power-up	Animated splash (`do_splash()`); any button skips it.
`idle`	default	Renders the current view; encoder tunes; handles held-button combinations.
`menu`	Menu button	`main_menu()` and sub-menus.
`recall`	Encoder push	`memory_recall()` directly.
`memory_scanner` / `frequency_scanner`	Back button past the last view	`memory_scan()` then `frequency_scan()`.
`sleep`	display timeout	OLED and TFT powered down until any input.

6.2 Controls in the idle state

Action	Effect
Turn encoder	Tune by the current step (`step_sizes[]` in settings.h).
Menu held + turn / Back held + turn / both held + turn	x10 step / step / 10 / x100 step.
Encoder push held + turn	Volume (0-9).
Encoder push + Menu held + turn	Mode (6 modes).
Encoder push + Back held + turn	Squelch (13 levels).
Back (short press)	Next view (0-7), then scanners.

Frequency wraps between the channel's `min_frequency` and `max_frequency`. When settings change the UI calls `apply_settings(false)` and `autosave()`.

6.3 Display views

Selected by `settings.global.view` in the switch at the end of the idle branch of `do_ui()`; `num_display_options` (= 8) must match the number of cases.

#	Function	Content
0	`renderpage_original`	Frequency, mode, S-meter, small spectrum.
1	`renderpage_bigspectrum`	Full-screen spectrum.
2	`renderpage_combinedspectrum`	Spectrum + waterfall.
3	`renderpage_waterfall`	Waterfall.
4	`renderpage_oscilloscope`	Audio waveform (from `get_audio_capture`).
5	`renderpage_status`	Diagnostics: battery, temperature, CPU busy, USB buffer, tuning offset.
6	`renderpage_smeter`	Analogue S-meter.
7	`renderpage_fun`	Novelty display.

Drawing helpers: `display_print_str/num/freq`, `display_set_xy`, `draw_spectrum`, `draw_waterfall`, `draw_analogmeter`, `draw_slim_status`. Pixels are written into the u8g2 frame buffer (`disp.buffer =`

`u8g2.tile_buf_ptr`) and pushed with `display_show()`. The OLED is SSD1306 or SH1106 (`oled_type` adds a 2-pixel x-offset) on I2C1, GPIO18/19, 400 kHz.

6.4 Menu system

Menus are built from a few generic widgets. A list of options is a single string where `#` separates items and `\n` splits a label onto two lines:

Widget	Use
`menu_entry(title, "A#B#C#", &selection, ok)`	Scrollable list; returns index.
`enumerate_entry(title, options, uint8_t &value, ok, changed)`	Pick one of N options into a setting.
`bit_entry(title, "Off#On#", bool &value, ok)`	Boolean setting.
`number_entry(title, fmt, min, max, multiple, value, ok, changed)`	Numeric setting (int8/uint8/int32 overloads).
`frequency_entry(title, uint32_t &f, ok)`	Digit-by-digit frequency editor.
`string_entry(char[], ok, del)`	Text entry (memory labels).

Menu tree:

```
Main: Frequency | Recall | Store | Volume | Mode | AGC | AGC Gain | Bandwidth | Squelch |
      Squelch Timeout | Noise Reduction >(Enable, Noise Estimation, Noise Threshold) |
      Impulse Blanker | Auto Notch | De-Emphasis | Bass | Treble | IQ Correction |
      Spectrum >(Spectrum Zoom, Spectrum Smoothing) | Aux Display | Band Start | Band Stop |
      Frequency Step | CW Tone Frequency | USB Stream | HW Config >
HW Config: Tuning Options | Display Timeout | Regulator Mode | Reverse Encoder |
      Encoder Resolution | Swap IQ | Gain Cal | Freq Cal | Flip OLED | OLED Type |
      Display Contrast | TFT Settings | TFT Colour | TFT Invert | TFT Driver |
      Bands >(Band 1..7) | IF Mode | IF Frequency | External NCO | USB Upload | Watchdog Test
```

Each menu maps `menu_selection` (the index in the option string) to a `case` in a `switch`. Most items call `apply_settings(false)` when `changed` so the effect is heard immediately while scrolling.

7. Settings and persistence

7.1 Settings structures (settings.h)

- `s_channel_settings` (per channel, also stored in memories): frequency, max/min frequency, mode, agc_setting, agc_gain, step, bandwidth.
- `s_global_settings`: everything else (volume, squelch, display, TFT, calibration, band thresholds, IF, noise reduction, view, tone, USB stream, encoder options, external NCO, some CW/TX placeholders).
- `default_settings`: the factory defaults, a positional aggregate initialiser. Its entries must stay in the same order as the struct members.
- `step_sizes[]` / `steps[]`: tuning step values and labels (13 entries).

7.2 Flash storage

Region	Declaration	Layout / behaviour
Auto-save	`autosave_memory[512][32]` (autosave_memory.cpp), 4 KB aligned, `__in_flash()`	512 slots of 32 words. Each save writes the next blank slot (first word 0xFFFFFFFF); when all are used the whole region is erased. Restore picks the last non-blank slot. `s_settings` must fit in 128 bytes (static_assert).
Memory channels	`radio_memory[512][16]` (memory.cpp)	512 channels of 16 words: `s_channel_settings` + 16-character label. Store rewrites the containing 4 KB sector. Factory contents are the initialiser in memory.cpp.

Every flash write uses the same safe sequence (in settings.cpp): suspend the receiver via `apply_settings_to_rx(..., suspend=true, ...)`, wait 10 ms, `multicore_lockout_start_blocking()` (core 1 called `multicore_lockout_victim_init()`), disable interrupts, erase/program, then reverse the steps. Reuse this pattern for any new flash data.

7.3 Memory loader tools (memory_loader/)

`memory.csv` lists channels as: Title, Frequency, Band Start, Band End, Mode, AGC Speed, Frequency Step, Bandwidth. `upload_memory.py <file.csv>` sends each channel over the USB serial port as a CAT command `ZUPnnn<16 hex words>;` while the radio runs normally; `download_memory.py` reads channels back with `ZDN`. `channel_to_words.py` packs a row into 16 words and mirrors `s_channel_settings`; the source carries the author's reminder to change this script whenever that struct changes. `memory.py` regenerates the factory `memory.cpp`. (HW Config > USB Upload is different: it reboots the Pico into its UF2 bootloader for a firmware update via `reset_usb_boot()`.)

8. Hardware interfaces

8.1 Pin map (pins.h)

GPIO	Name	Function	GPIO	Name	Function
0, 1	PIN_NCO_1/2	Quadrature LO (PIO0)	16	PIN_AUDIO	PWM audio out
2, 3, 4	PIN_BAND_0..2	Band filter select	17	PIN_BACK	Back button
5	PIN_ENCODER_PUSH	Encoder push	18, 19	DISPLAY_SDA/SCL	I2C1 OLED (+Si5351)
6, 7, 9	PTT / DIT / DAH	TX inputs (reserved)	20, 21	PIN_AB / PIN_B	Rotary encoder (PIO1)

GPIO	Name	Function	GPIO	Name	Function
8, 11, 13, 14, 15	MISO/DC/CS/SCK/MOSI	SPI1 TFT	22	PIN_MENU	Menu button
10, 12	MAGNITUDE/RF (TX_I/TX_Q)	Held low in RX	23	PIN_PSU	SMPS power-save (high)
25	LED	On-board LED	26, 27	ADC_I / ADC_Q	I/Q ADC inputs
28	PIN_MIC	Mic (reserved)	29	PIN_BATTERY	Battery ADC (ch 3)

A few pins are also hard-coded outside pins.h: `OLED_I2C_SDA_PIN/SCL_PIN` defines in ui.cpp, PIO NCO pins 0/1 in `nco.pio` (`nco_program_init`), and ADC GPIOs 26/27/29 and PSU pin 23 in the `rx` constructor. Change these together when remapping hardware.

8.2 Peripheral resource usage

Resource	Used by
PIO0 SM (claimed)	Quadrature NCO
PIO1 SM0	Rotary encoder (`quadrature_encoder.pio`)
DMA x5	ADC ping, ADC pong, capture, PWM audio ping, PWM audio pong (all claimed dynamically); DMA_IRQ_0 for ADC re-arm
ADC	Inputs 0/1 round robin (I/Q), 3 (battery), 4 (temperature)
PWM slice of GPIO16	Audio
I2C1	OLED via u8g2, optional Si5351
SPI1 @ 75 MHz	ILI934x TFT
USB	TinyUSB composite: audio source + CDC serial (stdio / CAT / memory upload)
Watchdog	2 s, fed from core-0 loop

8.3 Inputs

`button` (button.cpp) provides `is_pressed()` (short press event) and `is_held()`, updated at 20 Hz. `encoder` is an abstract base with `get_change()`; `rotary_encoder` reads a PIO quadrature counter and honours `reverse_encoder` / `encoder_resolution`; `button_encoder` (compile option `BUTTON_ENCODER`) emulates rotation with buttons.

8.4 Auxiliary TFT and SSTV (waterfall.cpp, ili934x.cpp, codecs/)

`waterfall::update_spectrum()` runs at 20 Hz and, depending on `aux_view`, draws a 120-line waterfall with frequency scale and S-meter or runs `decode_sstv()`, which pulls raw I/Q from the DSP queue via `rx::get_raw_data()` and feeds `c_sstv_decoder::decode_iq()`. Display type (ILI9341 / ILI9341_2), rotation, colour and inversion come from `configure_display()`.

8.5 CAT control (cat.cpp)

`process_cat_control()` accumulates USB-serial characters in a 256-byte circular buffer, extracts commands terminated by `;` and dispatches on the two-letter prefix with `strncmp`. Commands handled: FA, FB, SM, MD, IF, ID, FR, FT, AI, AG, XT, RT, RC, FL, PS, VX, RS, AC, SL, SH, IS, PR, NB, LK, MG, PL, VD, VG, BC, ML, NR, SD, KS, EX, RL, SQ, RG, RM, PA, RA, GT, PC, FW, TX, ZUP, ZDN. Setting commands modify `ui::settings` and call `apply_settings_to_rx()`.

9. Modification recipes

9.1 Add a new user setting that affects the DSP

- Add the field to `s_global_settings` (or `s_channel_settings` if it should be stored per memory channel) in settings.h.
- Add its default value to `default_settings` at the matching position.
- Add the runtime field to `rx_settings` in rx.h and copy it in `apply_settings_to_rx()` (settings.cpp).
- Add a setter to `rx_dsp` and call it from `rx::apply_settings()` (rx.cpp).
- Expose it in a menu (9.2). If it went into `s_channel_settings`, also update `memory_loader/channel_to_words.py` and the CSV format.
- Keep `s_settings` under 128 bytes and `s_memory_channel` under 64 bytes (static_asserts in settings.cpp).

9.2 Add a menu item

```
// ui.cpp, main_menu(): 1) append "My Item#" to the option string (before "HW Config#"
//    if you want HW Config to stay last), 2) add the matching case, renumbering if inserted:
case 25:
  done = enumerate_entry("My\nItem", "Off#Low#High#", settings.global.my_item, ok, changed);
  if(changed) apply_settings(false);
  break;
```

9.3 Add a display view

- Declare `void renderpage_myview(rx_status&, rx&)` in ui.h and implement it in ui.cpp using the existing draw helpers.
- Add a `case` to the view switch in `do_ui()` and increase `num_display_options`.

9.4 Add or change a demodulation mode

- Add a constant in rx_definitions.h (next ID 6) and to `e_mode` in settings.h.
- Add a branch in `rx_dsp::demodulate()` and a column in the `stop_bins` table / sideband logic in `set_mode()`.
- Update the UI strings: `modes[][]` in ui.h, the Mode `enumerate_entry` option string, the `% 6u` wrap in `do_ui()`, CAT MD mapping, and `channel_to_words.py`.

9.5 Change filter bandwidths, AGC or squelch

Edit the tables in `rx_dsp::set_mode()`, `rx_dsp::set_agc_control()` and `rx_dsp::set_squelch()` respectively. The Python scripts in `simulations/` (e.g. `agc_factors.py`, `audio_filters_des.py`, `am_sync_des.py`, `deemphasis.py`) show how the existing constants were derived.

9.6 Add a CAT command

Add another `else if (strncmp(cmd, "XX", 2) == 0)` branch in `process_cat_control()`, modify `settings`, then call `apply_settings_to_rx(receiver, settings_to_apply, settings, false, true)`. Replies are written with `printf` (stdio over USB CDC). The Z-prefixed commands (`ZUP` / `ZDN`) are PicoRX-specific and carry memory-channel upload/download.

9.7 Change pins or hardware

Update pins.h plus the hard-coded locations listed in 8.1. For a different band-filter decoder, edit the ladder in `rx::apply_settings()`. For a different OLED, change the `u8g2_Setup_...` call in the `ui` constructor. For a different clock range, regenerate the `possible_frequencies[]` table (`simulations/clocks.py`).

9.8 Things that are easy to disturb

- Core-1 timing: one block of DSP must finish in about 4.3 ms. The status page shows the busy time; keep new per-sample code in `__not_in_flash_func` functions and in integer arithmetic.
- Semaphore discipline: core-0 writes to `settings_to_apply` belong between `receiver.access()` and `receiver.release()` (as `apply_settings_to_rx()` does).
- Core 0 must keep feeding the watchdog (2 s) - long loops in UI code need to return to `do_ui()`.
- Flash writes must follow the suspend / lockout / disable-interrupts sequence in section 7.2.
- Mode indices, step indices and option strings are parallel lists across several files - change them together.

10. Supporting material

Folder	Contents
`simulations/`	Python models of the DSP (CIC, FFT filter, noise reduction, AM sync PLL, de-emphasis, AGC, IQ imbalance) plus C++ test harnesses (`test_dsp.cpp`, `fft_filter_test.cpp`, `noise_reduction_test.cpp`) that can run DSP code on a PC against `test.wav`.
`grc/`	GNU Radio Companion flowgraphs for AM, AM-sync and SSB reference designs.
`utils/`	`font.py` / seven-segment font generator, `decode_screendump.py`, `autosave.py`.
`user_manual/`	RST source and PDF of the end-user manual, useful for the intended UI behaviour.
`PCB/`	KiCad projects: main receiver (QSD, preamp, low-pass, Pico), front/back panels, concept board, through-hole front end.
`enclosure/`	FreeCAD model and STLs for a breadboard-version case (with/without TFT).

Appendix A. Q&A: Pico D-to-A conversion

Question: What is the highest sampling rate supported by the Raspberry Pi Pico D to A converter?

Answer: The Raspberry Pi Pico has no D-to-A converter. Neither the RP2040 (Pico) nor the RP2350 (Pico 2) includes a DAC, so there is no fixed maximum rate to quote. Analog output is made in one of these ways:

1. PWM plus an RC low-pass filter

This is what PicoRX uses on GPIO16. The update rate and the resolution trade off against each other:

```
update rate = system clock / (wrap + 1)
```

- At 125 MHz, 8-bit (wrap 255) gives about 488 kHz and 10-bit gives about 122 kHz.
- PicoRX runs the PWM at 240 kHz, with the system clock between 150 and 200 MHz. That gives a wrap of about 625-833, or roughly 9.3-9.7 bits.
- You can gain some extra effective resolution with oversampling and noise shaping.

2. PIO driving a resistor ladder (R-2R) on several GPIOs

This is a parallel DAC that can update at tens of MHz. The practical limits are resistor matching (about 8 bits is realistic) and GPIO slew rate.

3. An external DAC

Examples are an I2S audio DAC such as the PCM5102, driven by PIO at 44.1-192 kHz, or an SPI DAC such as the MCP4822. Use this route when you need true audio-quality or precision output.

Related: the ADC

The ADC tops out at 500 kS/s (48 MHz / 96 clocks per conversion), shared across all channels in round-robin mode. PicoRX runs it at 480 kS/s (clock divider 100), which gives 240 kS/s each for I and Q.